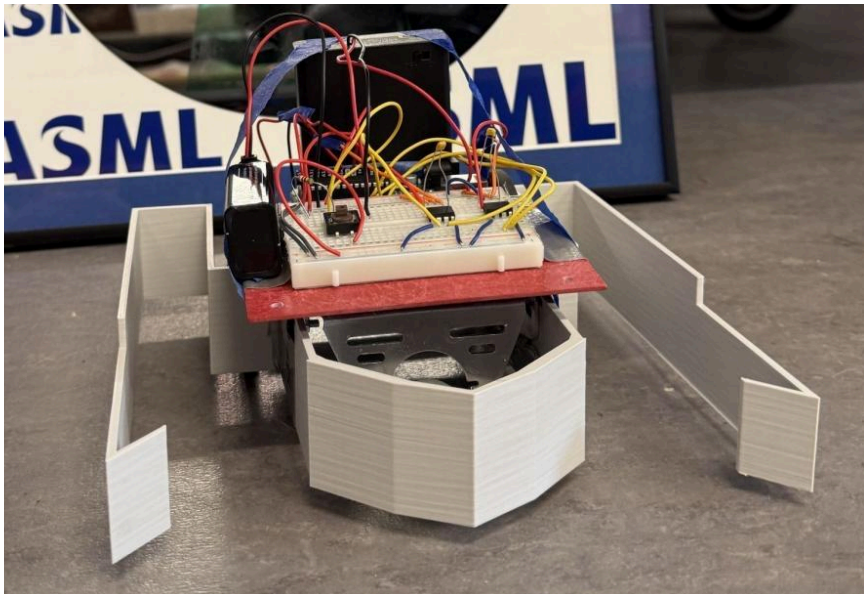
Group 62 - Team Clanker

Callum Coots (ckc86), Zachary Feldman (zlf3), Alexis Barrow (amb662)

1. Robot Design and Strategy Overview

The base strategy for our robot was simple. We prioritized speed above all else. While almost every other robot opted to use color/QTI sensors to patrol the arena and attempt to collect as many cubes as possible over the course of one minute, we recognized that pure speed would be able to counter the "common" strategy. So, instead of trying to collect every single cube, we aimed to reach the center of the arena before the opponent and collect just over half of the cubes in a very short time. We also recognized that since the cubes were consistently placed in the same region of the arena, and that we would not even be contacting the opposing robot, including sensors would be unnecessary. Instead, we hard-coded a movement path for our robot to follow every time that would sweep about 60% of the cube region and then end in a corner to minimize interference from the opponent (see appendix D). This strategy makes sense in theory, but how were we able to produce a meaningful speed advantage over the competition given the exact same components as everyone else? To do this, we had to get creative. We wired our 9V battery and 6V battery pack in series, and powered our arduino and motors with the resulting 15V power supply (see appendix B). We also removed all sensors to maximize current to our motors. The result was a robot that could move significantly faster than our opponents. Lots of testing and troubleshooting was conducted to minimize excess load on the motors and arduino, seeing as were were running them above rated voltage. For example, a 0.01 microfarad capacitor was placed across the terminals of each motor to smooth out voltage spikes and prevent brownouts. In addition to raw speed, we identified another method to cut time and reach the cubes faster. Most teams used the reset button on the arduino to start their robots. The issue is that this completely restarts the code, which takes about 1.5 seconds. To completely remove this 1.5 seconds of startup time, we implemented a pushbutton to start our motors (see Appendix B). Our code would be running prior to each round, and the pushbutton would trigger our motors to start, giving us an instant "launch," which proved to be a large advantage over our opponents. The final component of our robot was the cube-collection system. We aimed for the lightest, simplest design here to reduce load on our motors. We also avoided using any servos to control our arms, as this would draw current away from our main motors. To meet these criteria, we 3D-printed a collection piece that encompassed the entire 8"x8" maximum size (see appendix C). It included two static arms with slanted inlets to collect and retain cubes. It also utilized a triangle front, to direct cubes to either side of the chassis. Finally, our system covered the entirety of the chassis and wheels to improve durability and prevent cubes from getting jammed under the wheels or other components.



2. Design Process Reflection

Our design, prototyping, and testing process of the robot was highly dependent on the strategy our team decided to implement. Due to the price of our single 3D print taking up 99% of our budget, we couldn't afford to redesign our collection system, so we had to ensure the design worked. In the end, there were 7 different versions, all iterated on after design reviews. In terms of the central frame, too many of the components were near the front, causing a weight imbalance and the wheels would slip when turning, so we moved the arduino and the batteries toward the back to fix this issue. In the first design, the 6V battery pack was under the frame of the robot near the front and the arduino and 9V battery were on the top, near the front. To fix the weight balance issue, we moved the arduino to the back and taped the 6V and 9V batteries vertically in the rear, placing our center of mass as close to our two wheels as possible. Between the penultimate and final designs, we increased our effective voltage from 6V to 15V by wiring our 6V and 9V batteries in series. This demanded some electrical mitigation to prevent our motors from being damaged. A capacitor was added across the terminals of each motor in order to reduce the voltage spikes and prevent brownouts (see Appendix B). It took several re-designs to reach this solution and ensure our strategy would be successful. In fact, we were not able to even get our robot moving until midnight, the night before the competition. However, in the end, our robot was able to maintain its speed advantage while also remaining functional. In reflection, our design did not really follow the principles of milestone 3, as we used no sensors, but milestones 2 and 4 were highly relevant.

3. Competition Analysis

The robot performed extremely well, placing first in the competition. Our strategy proved to be fruitful, and other than three draws (two in the final match) and losing to the ASML robot, our robot went undefeated within competition, with a final record of 11-0-3.

The first draw was due to a specific failure that occurred: one of the wheels failed to move. When the robot was activated, the left wheel operated as it should have, while the right failed to respond in any capacity. This caused a large hiccup within the competition: as there was only a 5-round buffer to solve the issue. It ultimately took 2 rounds to correct the problem. There were multiple steps to uncovering the issue. The first approach was to check the physical components: done by exchanging the left and right-side components. This test was unsuccessful. Next was checking the right motor functionality through running 9V straight through it, which resulted in proving the motor was functional by itself. Ultimately the issue was identified as the connection of the motor wire, which was spliced to increase its length. A lack of secure connection caused a lack of power being given to the motor, which in turn caused the wheel to not be driven. To mitigate this problem, we removed the tape securing the two wires together, and proceeded to wrap the wires around each other more securely, before eventually securing it again with tape. This solution ultimately remedied the problem. The strengths of the robot lie both in strategy and in build: having been hard-coded and designed simply, it was efficient and able to move quickly to its destination while being flexible to overcoming a wide range of opposing designs (see Appendix D). However, there is a notable disadvantage within the design: due to the lack of predictability of the robot's behavior, the level of success is dependent on the block layout completely, as the robot was designed to sweep once and therefore had one opportunity to gather as many blocks as possible. This means that for whatever reason if there are not enough blocks within the given necessary range of the two arms of the outer frame (see Appendix A), then the outcome of the match is solely determined by the performance of the competing robot.

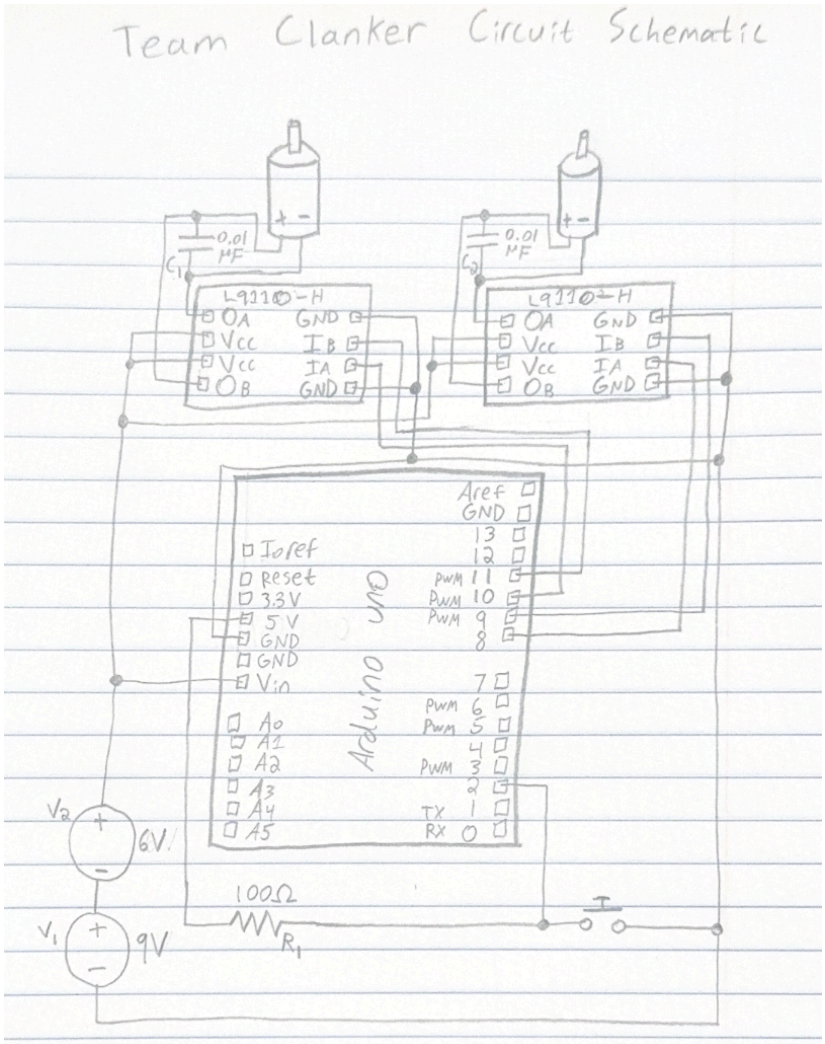
4. Conclusions

The robot was ultimately successful in fulfilling its intended purpose: being able to outmatch teams with its speed, trajectory, and cube-collecting mechanical design while not facing any challenges concerning victory against the opposing robots beyond the randomized cube setup prior to the start of the match. However, there had been a major issue during the competition concerning the right wheels' non-functionality mid-competition. Given the ability to redesign, this problem would be addressed. Through minimizing the amount of wires and components used on the circuit board and ensuring proper connection through intertwining any connecting wires, the robot would be less likely to fail due to a connection-integrity problem while allowing for faster debugging. By analyzing the team's performance through both setbacks and successes, future students would be able to draw both inspiration and caution from the robot's design. More specifically, a lesson can be learned on the design and build process. We would advise focusing on specific strategy rather than being as complex or hyper-efficient as possible. Through attempting too complicated of a design with too many strategies involved, the ability to focus on being efficient at achieving the specific goal of block collection becomes more difficult, as with more aspects of a design there are more areas of improvement and a distraction from the ultimate goal.

Appendix A

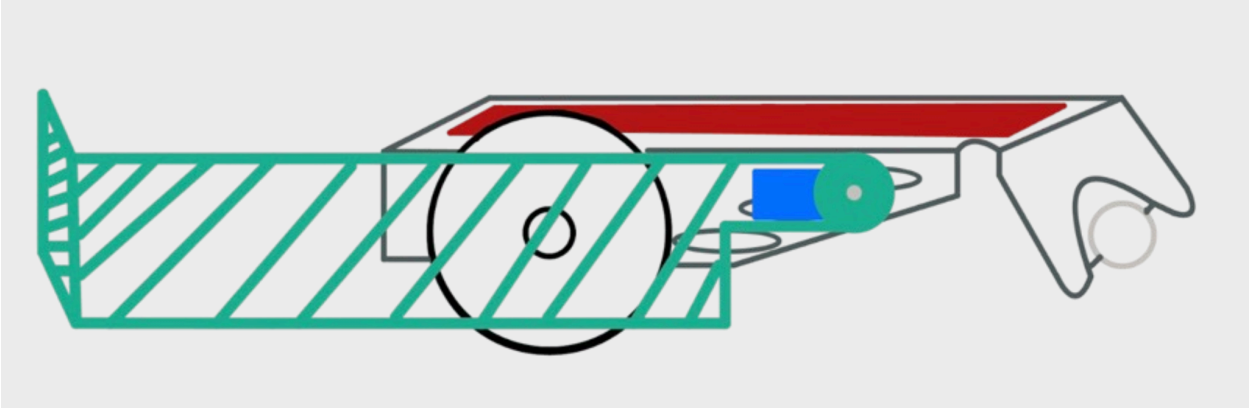
PLA 3D Print:
Weight = 95.28 grams (weight estimation is below)
Cost: 95.28(.4) + 1 = $39.11

Appendix B

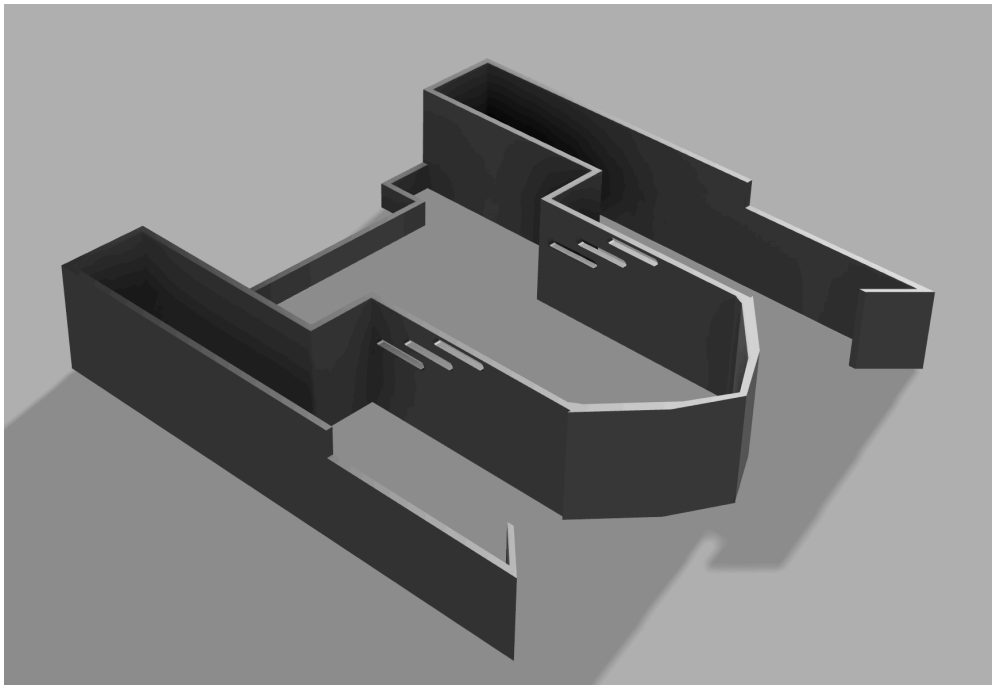


Appendix C

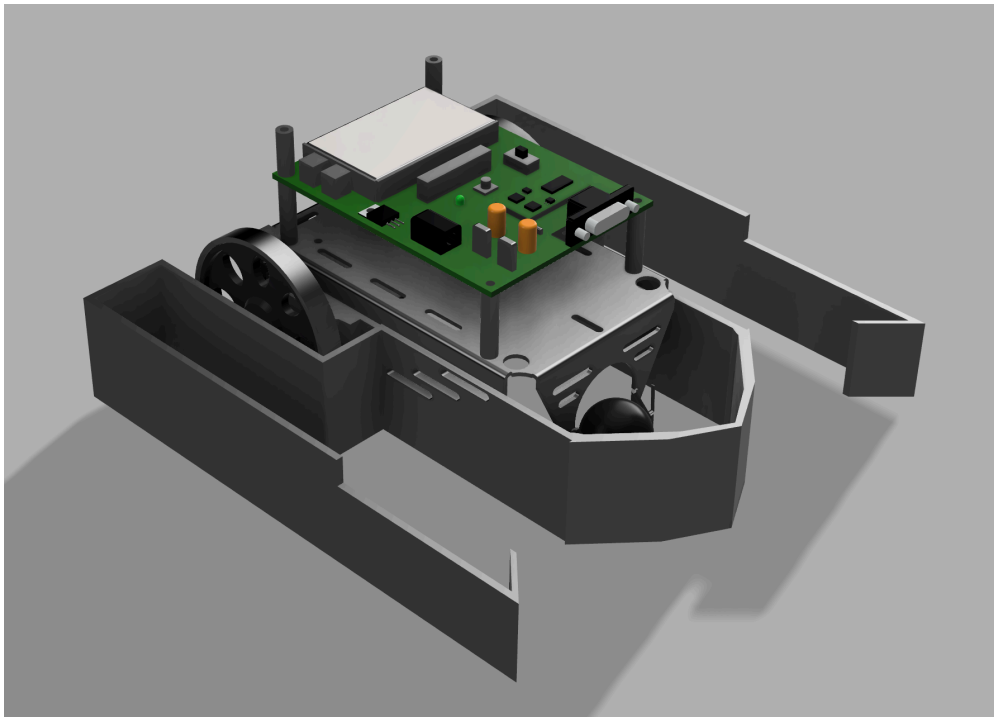
Preliminary Drawing:



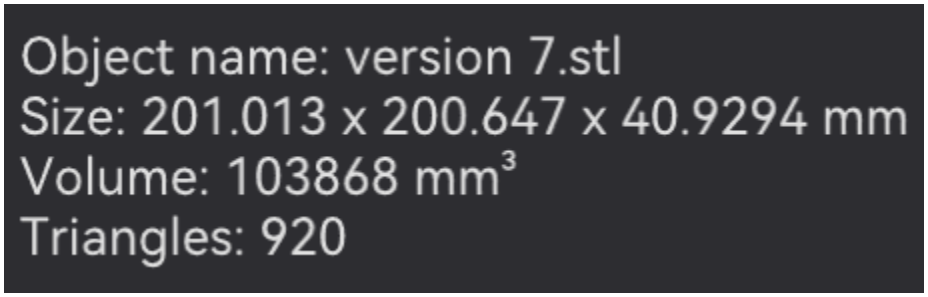
Intake System CAD Model:



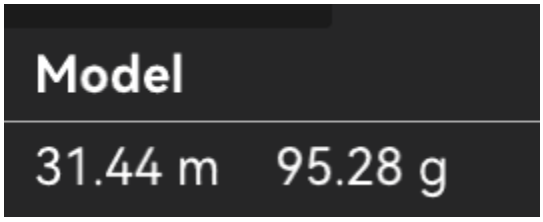
Whole Robot CAD:



3D Print Dimensions:

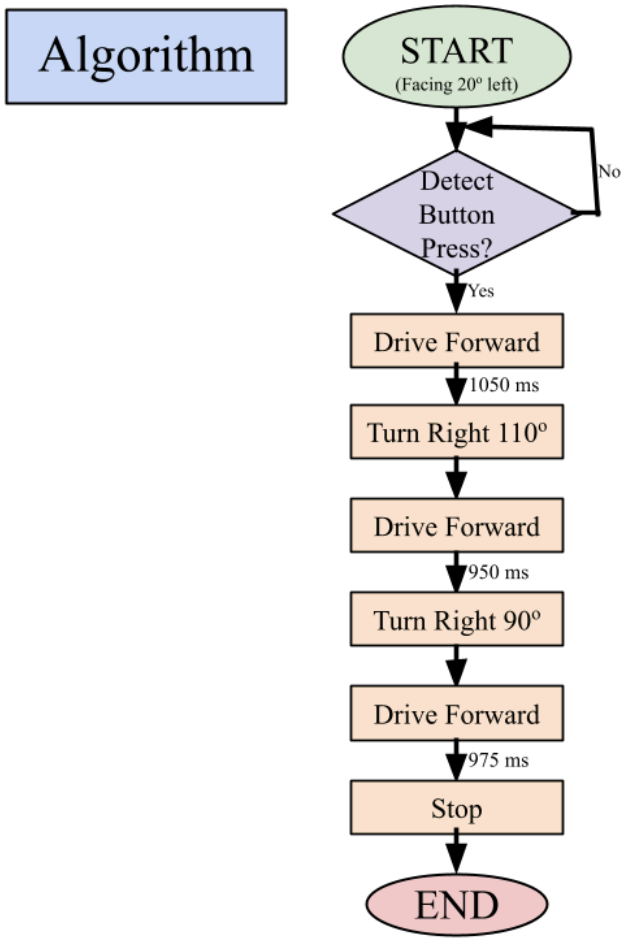


Weight Calculations for 3D Print from Bambu Slicer:



Print was also weighed and was the same weight within under 1%.

Appendix D



Appendix E

```
#include <avr/io.h>   //input definitions for input and output registers
#include <util/delay.h>   //include delay function for _delay_ms()

//declare all movement functions for later use
void front(void);
void back(void);
void right(void);
void left(void);
void stop(void);
void wideright(void);
void wideleft(void);

int main(void) {
DDRB |= 0b00001111; // PB0-PB3 outputs
DDRD = 0b00000000; // Port D inputs
```

```
while (1) {
//button start system (code runs before robot countdown and robot starts
on button press)
if (!(PIND & 0b00000100)){    //if pin D3 detects a low input, run
movement path

//Hard coded movement path is below

front();      //move forward
_delay_ms(1050);   //for 1.05 seconds

wideright();   //turn right
_delay_ms(745);   //~110 degrees

front();      //move forward
_delay_ms(950);   //for 0.95 seconds

wideright();    //turn right
_delay_ms(800);   //90 degrees (the delay is higher than before but this
is what worked in testing)

front();       //move forward
_delay_ms(975);    //for 0.975 seconds

stop();        //stop motors
_delay_ms(100000);   //for 1.67 minutes (enough time for the match to end)
  }
}
}

//define all movement functions for use in the movement path above
void front(void) {    //move forward
PORTB = (PORTB & 0b11110000) | 0b00001001;  //ports B4 and B1 high = both
motors forward
}

void back(void) {     //move backward (unused in final path)
PORTB = (PORTB & 0b11110000) | 0b00000110;  //ports B3 and B2 high = both
motors backward
```

```
}

void left(void) {     //turn sharp left (unused in final path)
PORTB = (PORTB & 0b11110000) | 0b00001010;  //ports B4 and B2 high = right motor forward, left motor backward
}

void right(void) {    //turn sharp right (unused in final path)
PORTB = (PORTB & 0b11110000) | 0b00000101;  //ports B3 and B1 high = right motor backward, left motor forward
}

void stop(void) {    //stop motors
PORTB = (PORTB & 0b11110000);  //all ports low = both motors stopped
}

void wideright(void) {    //turn right
PORTB = (PORTB & 0b11110000) | 0b00000001;  //port B1 high = right motor stopped, left motor forward
}

void wideleft(void) {    //turn left (unused in final path)
PORTB = (PORTB & 0b11110000) | 0b00001000;  //port B4 high = right motor forward, left motor stopped
}
```